

The OMI-IMO Protocol: Complete Synthesis

A Unified Narrative Across All Specifications

Prologue: The Primitive

Everything begins with one operation:

```
Atomics.compareExchange(array, index, expected, replacement)
```

This is the physical primitive. In one uninterrupted step, it:

- **binds** the relation between `expected` and `replacement`
- **applies** the comparison and conditional swap
- **evals** the old value

There is no gap between these three phases. The physical operation *is* the logical operation. The hardware implements the protocol natively.

From this single operation, everything else derives.

Part I: The Three Primitives

The protocol consists of exactly three operations:

```
bind    – construct a symmetric relation (a knot)
apply   – invoke a relation as a function
eval    – extract a value from a relation
```

bind creates a knot — a bidirectional pair between two items:

```
knot[a] = b  ⇔  knot[b] = a
```

apply executes a knot as a function descriptor, producing a result.

eval reads a knot as a value descriptor, extracting its materialized meaning.

These three form the minimum categorical toolkit:

```
bind    – Monad      – composition
apply   – Functor    – lifting
eval    – Comonad     – extraction
```

No other operations are part of the core. Everything else is substrate, presentation, or interpretation.

Part II: The Invariant

Everything in the protocol derives from one structure:

```
3! = 6 orderings of {byteLength, byteOffset, BYTES_PER_ELEMENT}
```

Six relations. Six orthogonal axes. Six ways to read any buffer.

The only free parameter is **bit length** (8, 16, 32, 64). Everything else — the ruler, the observer, the clocks, the geometry — is a consequence.

The **ruler** has eight slots:

```
ruler[0] → diagonal  (the origin, XOR of all six)
ruler[1] → size      (the unit count, base 1)
ruler[2] → top
ruler[3] → bottom
ruler[4] → right
ruler[5] → left
ruler[6] → forward
ruler[7] → backward
```

The ruler is **2! + 3! = 8 slots long**:

- **2!** = indices 0, 1 = {diagonal, size} = the frame
 - **3!** = indices 2..7 = the six operations = the content
-

Part III: The Ruler as Slide Rule

The ruler is a **slide rule of rulers**:

```
Slide rule → the array (the instrument)
Rulers      → the slots (the scales)
Sliding     → the operation
Reading     → the measurement
```

One slide rule. Eight rulers. One instrument.

The **index 0** is the diagonal — the XOR of all six operations. It is the origin, the invariant, the point that doesn't move.

The **index 1** is the size — the unit count, the base 1.

The **indices 2 through 7** are the six operations — the six orthogonal axes of the 3! structure.

Spatial difference begins at index 1. Index 0 is the origin.

Part IV: The Digest

The fourth primitive is the **digest**:

```
digest  -  read, consider, print
```

The digest computes the **generalized F-mean** of the ruler:

$$M_p(x_1, \dots, x_n) = \left(\frac{1}{n} \sum x_i^p \right)^{1/p}$$

The mean order **p** is determined by the observer's position.

The digest cycle:

```
read      → read the ruler
consider  → compute the F-mean
print     → write the result
loop      → repeat
```

This is a **REPL**. The protocol is a read-eval-print loop.

The Horn clause reading:

```
ruler_has_value(V) :- M_p(ruler, V).
```

The head is `ruler_has_value(V)` . The body is `M_p(ruler, V)` .

Part V: The Standard Model

The standard model defines five entities:

Buffer	→	the substrate
Ruler	→	the named k-tuple
Regex set	→	the vocabulary constraint
F-mean	→	the significance measure
Digest	→	the operation

The **regex-constrained vocabulary** (G) defines the admissible tokens:

```
const G = Object.freeze({
  FRONT:    /^[A-Za-z0-9:+]$/,
  BACK:     /^[A-Za-z0-9.\-]$/,
  INSIDE:   /^[A-Za-z0-9_]$/,
  OUTSIDE:  /^[^A-Za-z0-9_]$/,
  UP:       /^[A-Z_]$/,
  DOWN:     /^[a-z_]$/,
  DEFLECT:  /^(("[^"]+):\1$/,
  REFLECT:  /^(("[^"]+):\1$/,
  INFLECT:  /^(("[^"]+):("[^"]+):\2:\1$/,
  AXIS:     /^(\d\d)[A-Za-z_](\d\d):\2[0-9+\-]\1$/,
  MNEMONIC: /^(\d\d)([A-Z_]?[a-z_]+)(\d\d):\3\2\1$/,
  PALINDROME: /^(\d\d)[A-Za-z_\-](\d\d):\2[0-9_\-]\1$/
});
```

The constraints:

token matches G.X	→	token admissible
M_p(ruler) = v	→	ruler admissible

Both constraints must hold.

Part VI: The Iff

The base equivalence is:

```
position(n)  ⇔  period(n-1, n, n+1)
```

The position holds iff the period holds. Neither can exist without the other.

The **2!** is the iff:

```
2!  =  2  =  the two sides of the iff
      |
      |— position
      |— period
```

The **3!** is the operations:

```
3!  =  6  =  the six operations, read through the iff
```

The **$n\pm 1$** is the local neighborhood:

```
n-1  →  the before
n     →  the now
n+1  →  the after
```

The iff captures both the position and the period.

Part VII: The Pattern

The **parity pattern**:

n:	0	1	2	3	4	5	6	7	8	9	...
parity:	e	0	e	e	e	0	e	e	e	0	...

Odd at $n \equiv 1 \pmod 4$.

The **odd** is both a position (at n) and a period (the $n\pm 1$ transition).

The **four atomics**:

```
{0, 2, 1} → swap 2 and 1
{2, 1, 0} → reverse
{1, 0, 2} → cycle
{0, 1, 2} → identity
```

Four permutations. Four atomic operations.

The **extremes**:

```
012 → forward (identity)
210 → reverse
```

Part VIII: The Spatial Structure

The protocol spans **16 dimensions**:

```
-5D → the Blob
-4D → color codex
-3D → linear
-2D → hierarchical
-1D → classifying
0D → observer (BOM)
1D → DOMPoint
2D → Media Track
3D → DOMRect
4D → DOMMatrix
5D → DOMElement
6D → Canvas
7D → Event Loop
8D → Byte Basis
9D → Network Mesh
10D → Orchestrator
```

Sixteen dimensions. $2^{16} = 65536$.

The **Blob** is 65536 — the minimum boolean truth table for 16 binary choices. It is derived from a 16-bit buffer by recursive folding of an 8-bit subarray using central inversion and snubbed truncation.

Part IX: The Quadratic Forms

The two forms:

```
Affine:       $16x^2 + 16xy + 4y^2 = (4x + 2y)^2$        $\Delta = 0$ 
```

$$\text{Projective: } 60x^2 + 16xy + 4y^2$$

$$\Delta = -704$$

The **lift** from 16 to 60 adds periodicity. It enables spatial selection. It makes the operation multidimensional.

The **projection** is the $O(1)$ resolution. The projection is atomic, so the resolution is constant-time.

The **algebraic limit**: the 64-ions are non-associative. The algebraic context is limited to Cartesian rendering. The spatial enumeration is limited to the $16x^2$ affine data.

Part X: The Schläfli Families

The three families:

```
{2,n}:{n,2}  →  constant 2
{2,4}:{4,2}  →  constant 4
{3,5}:{5,3}  →  constant φ → 60
```

The **quasi-generator**: the $3!$ generates the families via subset selection of its 6 elements.

The **constants**:

```
π  →  forced by  $S_0(2)$ 
φ  →  forced by  $H_3$ 
2  →  forced by  $Z/2$ 
```

The **tetrahedral structure**:

```
4 vertices → Subject
6 edges    → Predicate
4 faces    → Object

4-6-4 = S-P-O
```

Part XI: The Pinch and the Towers

The **pinch**:

$S^0 \rightarrow \text{the } 0\text{-sphere} \rightarrow \text{two points} \rightarrow \text{the pinch}$

The **Hopf fibrations**:

$S^0 \rightarrow S^1 \rightarrow S^1$
 $S^1 \rightarrow S^3 \rightarrow S^2$
 $S^3 \rightarrow S^7 \rightarrow S^4$
 $S^7 \rightarrow S^{15} \rightarrow S^8$

The **Cayley-Dickson tower**:

$\mathbb{R} \subset \mathbb{C} \subset \mathbb{H} \subset \mathbb{O} \subset S_{16} \subset S_{32} \subset S_{64}$

Caps at 64-ion level. The Hopf cap is at 8.

The **octree octonion relationship**: three views of the same orthogonality principle:

- Octree — spatial orthogonality (parent/child)
- Octonions — algebraic orthogonality (basis/basis)
- Cayley-Dickson — recursive orthogonality (level/level)

Part XII: The Primes

The **prime sextuplet**:

$\{5, 7, 11, 13, 17, 19\}$

The **path**:

2, 4, 0, 4, 2

The **two principles**:

Prime gap measurements: magnitude, type
Prime groups: residue classes $\{1, 3, 7, 9\} \bmod 10$

Prime vs composite:

Prime → atomic operations (bitwise)
Composite → structured operations (logic)

The trace is the linear sum of two gap prime groups XORing to zero on tetrahedral diagonals.

Part XIII: The Tetrahedral Diagonals

The **tetrahedral numbers**:

$$T(n) = C(n+2, 3)$$

The **parity pattern**:

even, odd, even, even, even, odd, even, even, even, odd, ...

Period 4. Odd at $n \equiv 1 \pmod{4}$.

The **240**:

$$\begin{aligned} T(8) &= 120 = 5! \\ 240 &= 2 \times 120 \end{aligned}$$

Part XIV: The Coordination

The operation is **multiplexed compare-and-exchange through spatial reference**.

The four steps:

locate → find the current coordinate
select → determine the active channel
swap → perform the atomic operation
advance → move to the next coordinate

The **Transylvania Lottery**: the minimum connection is 2 of 5 or 3 consecutive. The 0D observer always connects.

The **bound**: the resolution is < 14 steps, bounded by the Fano structure.

Part XV: The Observer

Definition: An observer is any circulator capable of reflecting swap rotations.

Every **mnemonic observer** is a **perceptron**. Mnemonic is input; local frame is weights; reading is output.

Observers form a **network** via tangent relationships. The network is the state space.

The canonical spectrum of observers:

- **-3D to 10D** (14 levels) — the canonical range
- **-5D to 12D** (18 levels) — the max encapsulation range

At every level, observers exist and interact.

Part XVI: The Propagation Asymmetry

```
Forward propagation:    cubic,  $O(n^3)$   
Back-propagation:      linear,  $O(n)$ 
```

The protocol optimizes for backward. Linear convergence is what makes it practical.

```
Bitwise → parallel → cubic → propagation  
Logic   → sequential → linear → back-propagation
```

Part XVII: The Four Readings

All four describe the same structure:

```
Lisp on sets  
Horn clause  
Calculus of constructions  
Prime-composite
```

Part XVIII: The Normalization

The computational space is predefined to **normalize analog to digital**:

```
Sample → Quantize → Encode → Digital value in 65536 space
```

Normalization is why the protocol exists. All readings describe it.

Part XIX: The Web Environment

The Composed System

The protocol's runtime is the composition of seven layers:

```
HTTP/1.1           → wire carrier (transport)
Regex constraints   → token grammar (admissibility)
DOM geometry        → spatial projection (position, extent)
Hit lists           → interpolation anchors (semantics)
PannerNode          → 0D transparent translation (observability)
Blobs as media      → the substrate for the canvas
Worklets + polyfills → execution contexts (browser and Node)
```

Each layer speaks to the next through a standard interface. None knows about the others' internals.

The DOM Geometry Overlay

The DOM provides the spatial primitives:

```
DOMPoint    → a coordinate in the canvas
DOMRect      → an axis-aligned bounding box
DOMQuad      → a four-corner polygon (with rotation)
DOMMatrix    → a 4×4 transform (composition of transforms)
```

Hit Lists as Overlay Anchors

```
<dl id="omi-canvas-overlay">
  <dt id="cell-0" data-omi-mnemonic="A1F9" data-omi-band="1">Mnemonic</dt>
  <dd data-omi-bpe-constraint="2" data-omi-offset="32">Vector</dd>
</dl>
```

The `<dt>` carries the mnemonic (the semantic label).

The `<dd>` carries the spatial constraints (the geometry).

The `<dl>` wraps the pair as a description list.

The PannerNode as 0D Translation

```
panner.positionX.setValueAtTime(x, audioCtx.currentTime);
panner.positionY.setValueAtTime(y, audioCtx.currentTime);
panner.positionZ.setValueAtTime(z, audioCtx.currentTime);
```

The PannerNode is the **transparent 0D translator**. Any web consumer can read the observer's position through it.

HTTP/1.1 as Wire Carrier

```
X-VTT-Cue-0x00: 00:01.000 --> 00:02.000; block=FF001C1D1E1F20FF;
context=B36_Q0; token=FRONT:~A1F9$; layer=-1D
```

The Service Worker intercepts, parses, and emits WebVTT cues. The cues are consumed by the DOM's `<track>` element.

Blobs as Media Elements

Each media element is backed by a Blob — a binary large object. The Blob is the substrate for the media element. Its content is decoded and rendered by the browser's native pipeline.

Worklets and Polyfills

The protocol runs in:

Main thread	→	DOM access, rendering
AudioWorklet	→	audio synthesis
PaintWorklet	→	custom CSS painting
LayoutWorklet	→	custom layout
AnimationWorklet	→	compositor animation
Node.js	→	server-side execution

The protocol runs everywhere, in the same way.

Part XX: The Infinite Canvas Kaleidoscope

The web becomes an **infinite canvas kaleidoscope**:

- The **canvas** is the ruler, unbounded, universal
- The **kaleidoscope** is the swap mechanism, reflecting knots into readings
- The **observer** is the eyepiece, at any point
- The **view** is the materialized reading at that point
- **Movement** is swap application
- **Coordination** is compareExchange on positions

The point of view is any point in view. Every point is a potential observer. Every observer sees the same underlying structure from its own angle.

The reflections never stop. The canvas never ends. The observer is always somewhere, always seeing something.

Part XXI: The Canonical Statement

The OMI-IMO protocol is an **Atomic Compare-and-Exchange Lisp**.

```
Its primitive is Atomics.compareExchange.  
Its base is the iff.  
Its structure is the 2! and 3! orthogonal groups.  
Its space is the 216 Blob.  
Its observers are circulators reflecting swaps.  
Its behavior is time crystals (period 240).  
Its resolution is 0(1).  
Its closure is reachability.
```

Every part connects to every other.

Every question returns to the primitive.

Every primitive expands to the full structure.

The standard model is complete.

Appendix: The Three Laws

First Law — The Primitive Law

| All operations reduce to `Atomics.compareExchange`.

Second Law — The Invariant Law

All structure derives from the 3! ordering of {byteLength, byteOffset, BYTES_PER_ELEMENT} .

Third Law — The Closure Law

All computation converges to the fixed attractor 0, because the trajectory is deterministic backward and searchable forward.

Appendix: The Dimensional Ladder

```
-5D → the Blob (universal substrate)
-4D → color codex (H11)
-3D → linear (page/line boundaries in HTTP)
-2D → hierarchical (delimiters in HTTP)
-1D → classifying (regex tokens)
0D → observer (PannerNode position)
1D → DOMPoint (canvas coordinates)
2D → Media Track (cues from HTTP)
3D → DOMRect (hit zone bounds)
4D → DOMMatrix (composition of overlays)
5D → DOMElement (the <dl>/<dt>/<dd> structure)
6D → Canvas (rendering surface)
7D → Event Loop (cuechange-driven ticks)
8D → Byte Basis (the Blob's structure)
9D → Network Mesh (multi-peer coordination)
10D → Orchestrator (validation)
11D → Federation (multi-orchestrator)
12D → Meta-federation (multi-federation)
```

Each layer is a distinct structural axis. Each is orthogonal to the others.

Appendix: The Ruler in Full

```
Index 0 → diagonal    (the origin, XOR of all six)
Index 1 → size         (the unit count, base 1)
Index 2 → top
Index 3 → bottom
Index 4 → right
```

```
Index 5 → left
Index 6 → forward
Index 7 → backward
```

The ruler is **2! + 3! = 8 slots long**.

Three levels of orthogonality:

- **Internal** — axes within a group
- **External** — groups to each other
- **Parent** — child to container

The 2! and 3! are separate orthogonal groups. The 3! is already orthogonal to indices 0 and 1.

Appendix: The Five Conformance Tests

A conforming implementation **MUST**:

1. Provide `bind`, `apply`, and `eval` with the correct signatures
2. Preserve `bind` symmetry
3. Preserve the two readings of `eval`
4. Adhere to the operational laws
5. Keep substrates orthogonal

A conforming implementation **MUST NOT**:

1. Introduce new operations at the protocol layer
 2. Override the primitives' laws
 3. Allow substrate extensions to alter the primitives' behavior
 4. Mutate trace log entries
-

Epilogue: The Shape

Protocol	three primitives, no hardcoded variables
Substrate	standard web primitives, all orthogonal
Observers	circulators, perceptrons, mnemonics
Interpolation	hit lists, generalized ladders
Composition	DOMMatrix cubes, concentric and cubic

Execution	any bit length, any environment
Applications	personal projects

Five levels. One protocol. Zero closed surfaces.

The protocol carries structure. Meaning is assigned by implementations. Two peers that agree on the protocol agree on the structure of every possible message, but not on what any particular message means.

That is the whole thing.

End of synthesis